ATHENA - Hands On Git & GitHub

Alain Piallat, François-Gabriel Sourbé

19 octobre 2026

BDTech



Installation de Git

Selon votre système d'exploitation

Windows :

- Télécharger sur git-scm.com
- Installer avec les options par défaut
- Utiliser le terminal **Git Bash** (installé avec Git) pour tout le TD

macOS :

- Ouvrir le terminal et taper : `xcode-select -install`
- Ou via Homebrew : `brew install git`

Linux :

- Ubuntu/Debian : `sudo apt install git`
- Fedora : `sudo dnf install git`

BDTech



Vérification de l'installation :

```
1 git --version
```

Configuration de l'identité :

```
1 git config --global user.name "Pseudo GitHub"
2 git config --global user.email "votre.email@ensea.fr"
```

Important

Cette configuration sera associée à tous vos commits ! Utilisez la même adresse mail que celle de votre compte GitHub.



Étapes à suivre :

- Créer un compte sur github.com
- Utiliser ou ajouter votre adresse mail universitaire (`@ensea.fr`)
- Activer les fonctionnalités GitHub Education (gratuit pour les étudiants)

GitHub Student Developer Pack

Accès gratuit à GitHub Pro et à de nombreux outils de développement professionnels !



Configuration SSH - 1. Génération et Agent

Selon votre système (Linux, macOS ou Windows PowerShell)

Étape 1 : Générer la paire de clés (Tous OS)

```
ssh-keygen -t ed25519 -C "votre.email@ensea.fr"
```

Appuyez sur **Entrée** pour accepter l'emplacement par défaut (et définissez une passphrase si souhaité).

Étape 2 : Démarrer l'agent SSH et ajouter la clé

```
# Linux / Git Bash
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# macOS (Terminal natif)
eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

BDTech



Configuration SSH - 2. Spécificités Windows

Via PowerShell (sans passer par Git Bash)

Étape 2 (Windows) : Activer l'agent SSH dans PowerShell

- Ouvrir **PowerShell en tant qu'administrateur** pour activer le service :

```
Set-Service -Name ssh-agent -StartupType Automatic
Start-Service ssh-agent
```

- Puis ajouter la clé (PowerShell normal ou admin) :

```
ssh-add "$env:USERPROFILE\.ssh\id_ed25519"
```

Astuce

Si vous n'avez pas les droits administrateur sur le PC pour démarrer le service Windows `ssh-agent`, utilisez simplement **Git Bash** avec les commandes Linux de la diapositive précédente !

BDTech



Configuration SSH - 3. Ajout sur GitHub

Copier la clé publique et tester la connexion

Étape 3 : Afficher ou copier la clé publique

```
# Linux / macOS / Git Bash (afficher dans le terminal)
cat ~/.ssh/id_ed25519.pub
# Astuce macOS (copie directe dans le presse-papier) :
pbcopy < ~/.ssh/id_ed25519.pub

# Windows PowerShell (afficher ou copier directement)
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub" |
    clip
```

Étape 4 : Ajouter la clé sur GitHub

1. Aller dans **Settings** > **SSH and GPG keys** > **"New SSH key"**
2. Coller la clé publique, donner un titre (ex : "Mon PC") et valider

Étape 5 : Tester la connexion (Tous OS)

```
ssh -T git@github.com
```



Le concept de Fork

Créer sa copie personnelle

Qu'est-ce qu'un fork ?

- Une copie personnelle d'un dépôt Git hébergée sur votre compte GitHub
- Permet de travailler librement sans affecter le projet original
- Pratique courante en développement collaboratif et Open Source

Action :

1. Aller sur la page GitHub du dépôt du TD github.com/Athena-BDTech/HandsOn-Github.git
2. Cliquer sur le bouton **"Fork"** en haut à droite
3. Votre copie personnelle est créée sur votre compte !



Clone du projet

Récupération en local

Cloner VOTRE fork :

```
1 git clone git@github.com:votre-pseudo/HandsOn-Github.
      git
2 cd td-git-debutants
```

Vérification des dépôts distants (remotes) :

```
1 git remote -v
```

Résultat

`origin` pointe vers VOTRE fork sur GitHub (lecture et écriture).

BDTech



Configuration upstream

Lien avec le dépôt original

Ajouter le dépôt original comme remote :

```
1 git remote add upstream https://github.com/Athena-
      BDTech/HandsOn-Github.git
2 git remote -v
```

origin (votre fork)

- Lecture + Écriture
- Vos modifications personnelles

upstream (dépôt original)

- Lecture seule
- Modifications du dépôt original



Exploration et compilation du code C :

```
ls -la
cat README.md
gcc src/calculator.c -o calculator
./calculator
```

Observation de l'état du dépôt :

```
git status
```

Problème : Pollution du dépôt

Le fichier exécutable `calculator` (ou `calculator.exe`) apparaît dans `git status` !
Les fichiers compilés (`*.o`, `a.out`, exécutables) ne doivent **jamais** être versionnés.



Créer un fichier .gitignore

Ignorer les fichiers générés dès le début du projet

Créer et éditer le fichier `.gitignore` :

```
nano .gitignore
```

Contenu à mettre dans `.gitignore` :

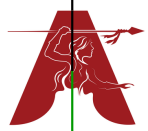
```
# Fichiers compilés
*.o
*.out
*.exe
calculator
a.out

# Autres fichiers à ignorer
*.log
```

Vérification :

```
git status
# Le binaire calculator n'apparaît plus, seul .
    gitignore est listé !
```





Créer un fichier de notes :

```
1 echo "# Mes notes du TD Git" > notes.md
2 git status
```

État "Untracked" (Non suivi)

Les fichiers `.gitignore` et `notes.md` existent dans le dossier mais Git ne les suit pas encore.

La commande magique :

```
1 git status
```

Utilisez-la **après chaque action** pour comprendre l'état de votre dépôt !



Le cycle add → commit → push

Les 3 étapes essentielles

1. Indexation (Staging) :

```
git add .gitignore notes.md
git status
```

2. Validation locale (Commit) :

```
git commit -m "Ajout du .gitignore et de mes notes
    personnelles"
```

3. Publication sur GitHub (Push) :

```
git push origin main
```



Avant d'indexer des fichiers modifiés :

```
1 echo "Git est très utile." >> notes.md
2 git diff
```

Bonne pratique

Toujours vérifier ses modifications avec `git diff` avant de faire `git add` !
N'oubliez pas ensuite de faire `git add notes.md` puis `git commit`.

Consulter l'historique des commits :

```
1 git log
2 git log --oneline --graph --all
```



Pourquoi des branches ?

Travailler en parallèle

Les branches permettent de :

- Développer une nouvelle fonctionnalité sans casser le code principal
- Tester des idées sans risque
- Travailler sur plusieurs tâches en parallèle
- Collaborer efficacement en équipe

Concept clé

La branche `main` (ou `master`) est la branche principale stable.
Les autres branches sont des "lignes temporelles parallèles".
On ne développe jamais directement sur la branche principale en production.

BDTech



Créer et basculer sur une branche

git switch vs git checkout

Méthode moderne (recommandée) :

```
1 git switch -c feature-multiply
```

OU méthode historique (équivalente) :

```
1 git checkout -b feature-multiply
```

Lister les branches et voir la branche active (*) :

```
1 git branch
```

Exercice

Créez et basculez sur la branche `feature-multiply` pour ajouter la fonction `multiply` à la calculatrice.



Développement sur la branche

Ajout de la fonction multiply

Modifier `src/calculator.h` :

```c
int multiply(int a, int b);
```

Modifier `src/calculator.c` :

```c
int multiply(int a, int b) {
    return a * b;
}

// Dans main()
printf("6 * 7 = %d\n", multiply(6, 7));
```

Tester et valider les modifications sur la branche :

```
gcc src/calculator.c -o calculator
./calculator
git add src/
git commit -m "Ajout de la fonction multiply"
```



Fusion de branche

git merge

1. **Retourner sur la branche principale `main` :**

```
git switch main
```

2. **Fusionner la branche `feature-multiply` dans `main` :**

```
git merge feature-multiply
```

3. **Visualiser l'historique et nettoyer la branche fusionnée :**

```
git log --oneline --graph --all
git branch -d feature-multiply
git push origin main
```



Qu'est-ce qu'un conflit ?

Quand Git ne peut pas choisir automatiquement

Un conflit survient quand :

- Deux branches (ou deux développeurs) modifient la **même ligne** d'un même fichier de manière différente
- Lors de la fusion (`merge` ou `pull`), Git ne peut pas décider seul quelle version garder

Règle d'or en travail collaboratif

- Toujours faire `git pull` **avant de commencer à travailler** et **avant de faire un** `git push` afin de détecter et résoudre les conflits en local.
- Éviter de travailler simultanément à plusieurs sur la même branche.

BDTech



Provoquer un conflit en local

Exercice pratique en autonomie

1. Créer une branche `style-titre` et modifier le titre :

```
git switch -c style-titre
# Dans src/calculator.c, remplacer le premier printf
    du main par :
# printf("=== Calculatrice en C ===\n");
git add src/calculator.c
git commit -m "Titre version C"
```

2. Revenir sur `main` et modifier la même ligne autrement :

```
git switch main
# Dans src/calculator.c, remplacer ce même printf par
    :
# printf("=== Calculatrice Avancee ===\n");
git add src/calculator.c
git commit -m "Titre version Avancee"
```

BDTech



Détecter le conflit

Tentative de fusion des deux branches

Fusionner `style-titre` **dans** `main` **:**

```
1 git merge style-titre
```

Conflit détecté !

```
Auto-merging src/calculator.c
CONFLICT (content): Merge conflict in src/calculator.c
Automatic merge failed; fix conflicts and then commit the
result.
```

Observer l'état avec `git status` **:**

```
1 git status
```



Résolution manuelle du conflit

Éditer le fichier en conflit

Ouvrir `src/calculator.c` — Git a inséré des marqueurs :

```
<<<<<<< HEAD
    printf("=== Calculatrice Avancee ===\n");
=======
    printf("=== Calculatrice en C ===\n");
>>>>>>> style-titre
```

Choisir une version (ou combiner les deux) et supprimer les marqueurs «««<, =======, »»»> :

```
    printf("=== Calculatrice Collaborative en C ===\n"
        );
```



Finaliser la résolution

Commit de fusion

1. Vérifier que le code compile toujours :

```
gcc src/calculator.c -o calculator
```

2. Marquer le conflit comme résolu avec `git add` :

```
git add src/calculator.c
git status
```

3. Finaliser la fusion et pousser :

```
git commit -m "Résolution du conflit sur le titre"
git branch -d style-titre
git push origin main
```



Commandes essentielles

Votre boîte à outils Git

État et historique :

```
git status
git log --oneline --graph
git diff
```

Cycle de base :

```
git add <fichier>
git commit -m "Message"
git push origin main
```

Synchronisation :

```
git pull origin main
git pull upstream main
git remote -v
```

Branches :

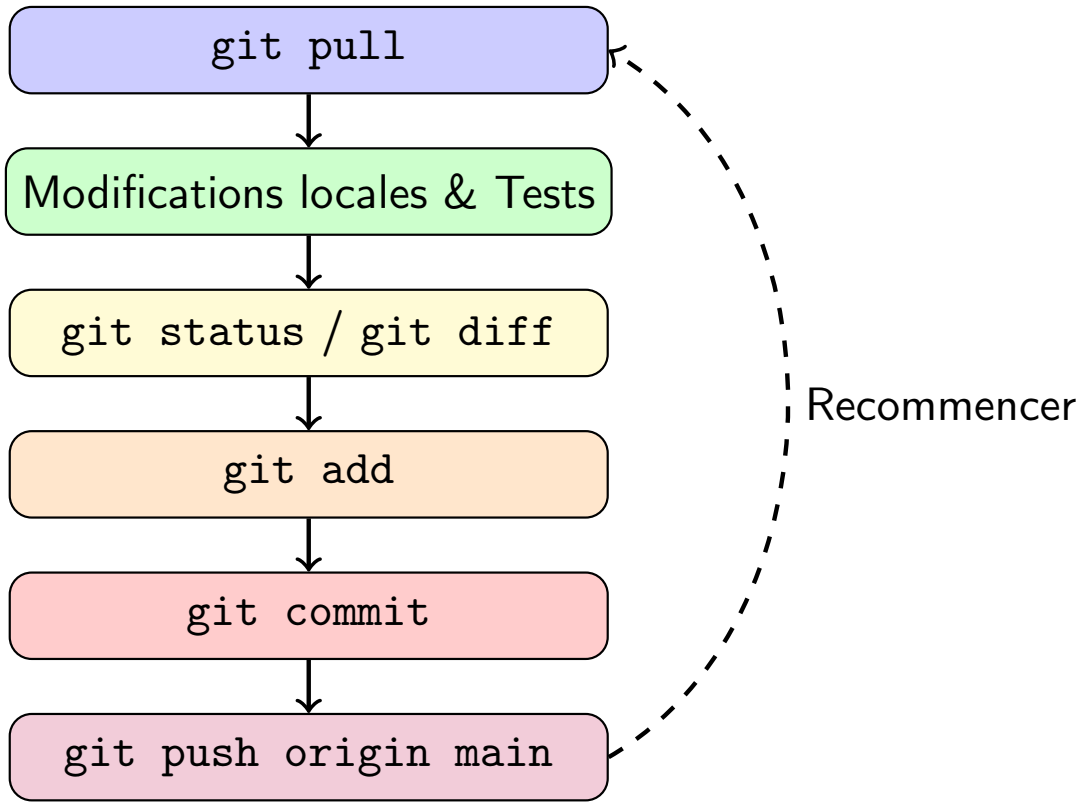
```
git switch -c <branche>
git switch main
git merge <branche>
git branch -d <branche>
```

BDTech



Workflow Git complet

Schéma récapitulatif



Bonnes pratiques

Conseils pour bien utiliser Git

- **Commit souvent** : Petits commits réguliers et atomiques plutôt qu'un gros commit
- **Messages clairs** : Décrire ce qui a été fait et pourquoi
- **Une branche = une fonctionnalité** : Isoler chaque développement
- **Pull avant de coder et avant de push** : Toujours se synchroniser pour gérer les conflits en local
- **.gitignore dès le début** : Ne jamais versionner les fichiers compilés ou sensibles
- **Tester avant de commit** : Vérifier que le code compile et fonctionne
- **git status est votre ami** : L'utiliser sans modération !

BDTech



Documentation du projet

README et Licence

Le fichier `README.md` :

- Présentation du projet
- Instructions d'installation et de compilation
- Guide d'utilisation
- Comment contribuer

La licence (`LICENSE`) :

- Définit le cadre légal et les droits d'utilisation/partage
- Licences open source populaires : MIT, GPL, Apache 2.0
- À définir dès le début du projet

BDTech



Pour aller plus loin

Ressources et pratiques avancées

Concepts avancés (hors TD) :

- **Pull Requests (PR)** : Proposer ses modifications du fork vers le dépôt `upstream` sur GitHub
- **Git rebase** : Réécrire et linéariser l'historique
- **Git hooks / GitHub Actions** : Automatisations et tests continus (CI/CD)
- **Git submodules** : Gérer des projets imbriqués

Ressources utiles :

- Documentation officielle : `https://git-scm.com`
- Exercices interactifs : `https://learngitbranching.js.org`
- Fiche mémo de GitHub : `https://education.github.com/git-cheat-sheet-education.pdf`

BDTech



Questions ?

Merci pour votre attention !

Des questions ?

Mail : athena@ensea.fr
Instagram : @athena_ensea

BDTech

